

HOW TO BECOME A

DANGEROUSLY OVEREDUCATED ENGINEER

The complete guide to self-educating your way
into internships you "shouldn't" qualify for

by @1nska

Engineering Student @ NYU Shanghai

■■ → Shanghai | Business → Engineering | Builder

BEFORE WE START

Why I Made This Guide

I switched from a business major to engineering because I wanted to build things with my own hands. No safety net. No guarantee it would work out.

Since then, I've worked with KUKA — one of the world's top robotics companies — on a robo art performance, started building a fintech startup from my dorm room, and learned more in 6 months of building than 2 years of lectures taught me.

This guide is everything I wish someone had handed me on day one. It's the exact framework I used to go from "business student who can't code" to someone who actually ships software. No fluff. No motivational quotes. Just the playbook.

If you follow even half of this, you'll be in the top 5% of engineering students — and internship recruiters will notice.

STEP 01

Stop Relying on Your Classes

Your degree teaches you the floor, not the ceiling. If you only learn what's in the syllabus, you'll have the exact same skills as every other graduate. That's not a competitive advantage — that's a commodity.

What to do instead:

- Use your classes as a starting point, then go 2–3 levels deeper on your own
- If a class covers data structures, go build a project that uses them in a non-trivial way
- If a class covers circuits, go replicate a real product's hardware
- Treat your professors as mentors, not just lecturers — go to office hours with real questions beyond the curriculum

The mindset shift:

Classes are a map. The territory is what you explore on your own. The students who land the best internships treat their coursework as 40% of their education — the other 60% is self-directed.

Free resources to go deeper:

- MIT OpenCourseWare — full MIT courses with problem sets and exams
- Stanford CS courses on YouTube — CS229, CS231n, CS106B
- 3Blue1Brown — visual math intuition that makes everything click
- The Feynman Lectures — physics fundamentals explained by a genius

STEP 02

Pick a Stack and Go Stupid Deep

Don't learn 10 languages at 10%. Learn one at 100%. Recruiters smell generalists from a mile away. The person who can say "I know Python deeply — here's a complex project to prove it" will always beat the person who lists 8 languages on their resume but can't build anything meaningful in any of them.

How to pick your stack:

- **If you want to build apps:** Learn Swift (iOS) or React Native / Flutter (cross-platform). Pair with a backend like Node.js or Python/Django
- **If you want to do AI/ML:** Python is non-negotiable. Learn PyTorch or TensorFlow deeply. Know NumPy and Pandas cold
- **If you want to do systems/infra:** C/C++ or Rust. Understand memory, concurrency, and operating systems at a low level
- **If you want to do web:** TypeScript + React + a backend framework. Learn it end-to-end so you can build full products solo

The depth test:

Can you build a complete, non-trivial project in your chosen stack without following a tutorial? If yes, you're deep enough. If no, keep going.

■ *Pro tip: Read the source code of libraries you use. Understanding how React or PyTorch works under the hood separates junior developers from engineers.*

STEP 03

Build Something Nobody Asked You to Build

This is the single most important step in this entire guide. I convinced KUKA — one of the world's top robotics companies — to let me borrow a robot for an art engineering performance. No budget. No connections. No one handed it to me. I just reached out, pitched the vision, and made it happen. That one project taught me more than any class and gave me something concrete to talk about in every conversation.

Why this works:

- It proves initiative — anyone can do homework, very few people build on their own
- It creates real stories for interviews ("Tell me about a project...")
- You learn 10x faster when you're solving real problems, not textbook exercises
- It gives you something to put on your resume that isn't a class project everyone else also has

What to build (start here):

- **Solve your own problem:** What annoys you? What's inefficient? Build a tool to fix it
- **Pitch something bold:** Reach out to companies, labs, or people way above your level. The worst they can say is no
- **Clone and improve:** Rebuild a product you use daily, then add a feature they don't have
- **Automate something:** Find a manual process and engineer it away

The bar:

Your project should be something you can demo in 60 seconds and explain why it matters in one sentence. It doesn't need to be perfect. It needs to be shipped.

■ *Ship > perfect. A finished ugly app beats an unfinished beautiful one every time.*

STEP 04

Read Documentation Like Other People Read Instagram

Seriously. The engineers who read docs outperform the ones who watch tutorials every single time. Tutorials teach you to follow instructions. Documentation teaches you to think.

Why docs beat tutorials:

- Docs are written by the people who built the thing — tutorials are written by people who learned to use it
- Docs are comprehensive — tutorials cherry-pick the easy parts
- Reading docs builds the skill of learning from primary sources — the #1 skill of senior engineers
- In interviews, saying "I read the RFC" signals a completely different level of seriousness

Docs to start with:

- **Whatever framework you use daily** — read it cover to cover, not just "Getting Started"
- **MDN Web Docs** — the bible for anything web-related
- **Apple Developer Documentation** — if you're building iOS/macOS
- **Python official docs** — especially the tutorial and library reference
- **RFCs and specs** — HTTP, TCP/IP, OAuth. Dense but incredibly rewarding

The habit:

Spend 20 minutes a day reading documentation instead of watching a tutorial. In 3 months, you'll understand systems at a level most students don't reach in 4 years.

STEP 05

Reverse Engineer Real Products

Contributing to open source or breaking down how real products work is the fastest shortcut to thinking like a senior engineer while you're still a student.

How to get started:

- **Pick a product you love** and figure out how it works technically. How does Spotify handle offline? How does Notion render blocks?
- **Read open-source code** of tools you use — VS Code, React, popular libraries in your stack
- **Start with "good first issue" labels** on GitHub. Fix a bug. Improve a doc. Submit a PR. The bar is lower than you think
- **Write technical breakdowns** of how things work and post them — this doubles as portfolio content

Why recruiters love this:

Open-source contributions show you can read other people's code (critical in any job), understand complex systems, and contribute to a codebase you didn't create. These are the exact skills you'll need on day one of any internship.

Where to find projects:

- GitHub Explore — github.com/explore
- Good First Issue — goodfirstissue.dev
- Up For Grabs — up-for-grabs.net
- CodeTriage — codetriage.com

STEP 06

Make Your Internet Presence a Portfolio

When a recruiter Googles you, they should find projects, not just a LinkedIn with your GPA. Your online presence is your 24/7 interview.

GitHub Profile

- Pin your 4–6 best repos with clear README files
- Each README should explain: what it does, why you built it, and how to run it
- Keep your contribution graph active — consistency signals commitment

LinkedIn

- Headline: not just "student at X" but what you build and your unique angle
- Feature your projects in the Experience section as if they were jobs
- Post technical content occasionally — build logs, lessons learned

Personal Website

- A simple portfolio: projects, short bio, contact info (GitHub Pages is free)
- Include case studies: problem → approach → result → what you learned

Social Media

- Document your build journey publicly — this is how I grew my audience
- Opportunities come to people who are visible. Recruiters, founders, and mentors are all watching

BONUS

The Internship Gameplan

If you've followed steps 1–6, you're already ahead of 90% of applicants. Here's how to convert that into actual offers.

3 months before applications:

- Have at least 2 shipped projects you can demo and explain in detail
- Have your GitHub, LinkedIn, and resume all telling the same story
- Start LeetCode / NeetCode prep — you still need to pass technical screens
- Make a target list of 20–30 companies, prioritized by fit, not prestige

During applications:

- Tailor every resume to the specific role — generic resumes get generic results
- Cold DM engineers at target companies on LinkedIn. Ask genuine questions first
- Apply to startups too — steeper learning curve and they care more about what you've built
- Your projects ARE your pitch. Lead every conversation with what you've shipped

In interviews:

- "Tell me about yourself" = tell them what you've built and why, not your GPA
- For behavioral questions, use your build stories — challenges, tradeoffs, failures
- Think out loud in technical questions. Nobody expects a student to know everything
- Ask smart questions back — about the stack, team, what you'd work on

■ *Internships don't go to the smartest students. They go to the ones who look like they don't need the internship to prove they can build. Be that person.*

RESOURCES

The Full Resource List

Learning Platforms

- MIT OpenCourseWare — ocw.mit.edu
- Stanford Online — online.stanford.edu
- 3Blue1Brown — youtube.com/3blue1brown
- freeCodeCamp — freecodecamp.org

Documentation

- MDN Web Docs — developer.mozilla.org
- Apple Developer — developer.apple.com/documentation
- Python Docs — docs.python.org
- React Docs — react.dev

Open Source

- GitHub Explore — github.com/explore
- Good First Issue — goodfirstissue.dev
- Up For Grabs — up-for-grabs.net
- CodeTriage — codetriage.com

Interview Prep

- NeetCode — neetcode.io (structured LeetCode roadmap)
- LeetCode — leetcode.com
- Blind 75 — curated list of 75 must-know problems
- system-design-primer (GitHub) — [system design fundamentals](https://github.com/donnydon/system-design-primer)

Portfolio & Hosting

- GitHub Pages — pages.github.com
- Vercel — vercel.com
- Notion — notion.so (project case studies)

What's Next?

This guide is your starting point, not the finish line.
The only thing that separates you from the people who land incredible internships is execution.

Follow **@1nska** for daily build logs, lessons, and the real story
of building a startup as a student.

Ship the code. Tell the story. Build in public.
The opportunities will come.